



Plain Text Files

Part 1: Manipulating Strings

Plain Text Files are made up entirely of strings. To use them effectively, you need to know how to manipulate strings effectively.

Plain Text Files Use String Values

Plain text files store information as text, so Python reads their contents as **strings**.

Even if a file contains numbers, those values initially behave like text until they are converted to another data type.

Understanding common string methods makes it easier to clean, separate, modify, and convert data read from files.

lower() / upper() / capitalize() / title()

These string methods change the capitalization of text.

lower() makes all letters lowercase, while **upper()** makes them uppercase

capitalize() capitalizes the first character, and **title()** capitalizes the first letter of each word.

These methods return a new string rather than changing the original string.

```
text = "hello WORLD"

print(text.lower())
print(text.upper())
print(text.capitalize())
print(text.title())
```

OUTPUT

```
hello world
HELLO WORLD
Hello world
Hello World
```

strip() / lstrip() / rstrip()

These methods remove whitespace from the beginning or end of a string.

strip() removes whitespace from both sides, while **lstrip()** removes it from the left and **rstrip()** removes it from the right.

They are especially useful when cleaning text read from a file.

```
text = "  hello  "

print(text.strip())
print(text.lstrip())
print(text.rstrip())
```

OUTPUT

```
hello
hello
  hello
```

In this example, the white space is highlighted in grey for emphasis.

split()

The `split()` method separates a string into a list of smaller strings.

Without a parameter, it normally splits the text wherever whitespace appears.

A separator can also be provided to split the string at a specific character or sequence.

```
# No separator used  
sentence = "red green blue"  
  
print(sentence.split())
```

OUTPUT

```
['red', 'green', 'blue']
```

split()

The `split()` method separates a string into a list of smaller strings.

Without a parameter, it normally splits the text wherever whitespace appears.

A separator can also be provided to split the string at a specific character or sequence.

```
# Uses , as a separator
data = "Alex,82,91"

print(data.split(","))
```

OUTPUT

```
['Alex', '82', '91']
```

replace()

The `replace()` method creates a new string where one piece of text has been replaced with another.

It can replace individual characters, words, or longer sequences of text.

The original string is not changed unless the returned value is stored.

```
message = "Hello Sam"
message = message.replace("Sam", "Alex")
print(message)
```

OUTPUT

Hello Alex

join()

The `join()` method combines multiple strings into one string.

It is called on the string that should be placed between each value.

This is useful for rebuilding text after it has been separated into a list.

```
words = ["red", "green", "blue"]  
  
text = ", ".join(words)  
  
print(text)
```

OUTPUT

```
red, green, blue
```

Casting to `int()` or `float()`

Numbers stored inside strings cannot be used directly for normal mathematical operations.

`int()` converts a string into an integer, while `float()` converts it into a decimal number.

This is commonly necessary when numeric data has been read from a text file.

```
age_text = "16"  
price_text = "12.50"  
  
age = int(age_text)  
price = float(price_text)
```